

Yesterday 9:19 PM

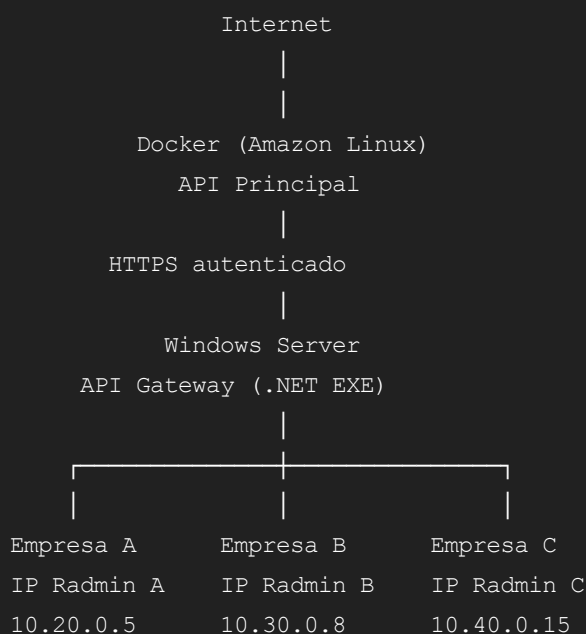
Só mais um TODO para fazer na API windows: Quando a API ~windows receber alguma requisição de alguma forma terá que entender de qual empresa se trata para direcionar qual IP tunel radmin irá utilizar, assim vou utilizar somente um executável para qualquer requisição.

Essa decisão muda bastante a arquitetura, e na minha visão é uma abordagem melhor do que manter um executável por empresa.

O problema que você está resolvendo é um problema clássico de **multi-tenant** (multiempresa).

Sua API Windows deixa de ser uma API "para uma empresa" e passa a ser um **Gateway de Integração**.

A arquitetura ficaria aproximadamente assim:



Fluxo

O cliente chama

POST /pedido

enviando algum identificador.

Exemplo

</> JSON



```
{  
  "empresa": "empresa01",  
  "pedido": 12345  
}
```

ou

</> http



POST /pedido

Header

X-Empresa: empresa01

A API faz:

Recebe requisição



Lê empresa



Consulta configuração



Descobre IP Radmin



Conecta no servidor correto



Executa operação



Retorna resposta



Eu criaria uma tabela de configuração

Pode ser até um JSON inicialmente.

</> JSON



```
{  
  "empresa01": {
```

```
{
  "ip": "10.20.0.5",
  "porta": 3050,
  "database": "C:\\Empresa01\\Dados\\db.fdb"
},

"empresa02": {
  "ip": "10.30.0.8",
  "porta": 3050,
  "database": "D:\\Empresa02\\Banco\\db.fdb"
}
}
```

Ou numa tabela SQL

EMPRESAS



ID

NOME

TOKEN

IP_RADMIN

PORTA

CAMINHO_BANCO

ATIVA

Assim a API não possui nada fixo.

Melhor ainda

Ao invés de confiar no nome da empresa enviado pelo cliente, eu utilizaria um **API Key**.

Exemplo:

```
Authorization:
Bearer 8DJKEFJDS89FDJ...
```



ou

```
X-API-Key:
98A73BC...
```



A API faria

API Key



Empresa 15



IP 10.20.0.8



Executa



O cliente nunca informa o IP.

Nunca informa a empresa.

Só a chave.

Vantagens

Você consegue:

- um único executável
- um único serviço Windows
- um único deploy
- adicionar empresa sem recompilar
- alterar IP Radmin sem publicar novamente
- trocar servidor apenas alterando cadastro

Eu ainda adicionaria cache

Na primeira chamada

Empresa 15



consulta banco

Empresa 15

IP 10.20.0.5



e guarda em memória por alguns minutos.

As próximas chamadas já usam o cache.

```
MemoryCache
```



```
Empresa15
```

```
↓
```

```
10.20.0.5
```

Isso evita consultar a tabela de configuração em toda requisição.

Uma preocupação importante

Se sua API puder executar várias requisições simultaneamente para empresas diferentes, evite usar qualquer variável global para armazenar o IP atual. Cada requisição deve resolver sua empresa e utilizar sua própria configuração durante todo o processamento.

O fluxo ideal é:

```
Requisição
```



```
|
```

```
▼
```

```
Identifica empresa
```

```
|
```

```
▼
```

```
Obtém ConfigEmpresa
```

```
|
```

```
|— IP Radmin
```

```
|— Porta
```

```
|— Banco
```

```
|— Credenciais
```

```
|
```

```
▼
```

```
Executa toda a operação usando esse objeto
```

Assim duas requisições concorrentes, uma da Empresa A e outra da Empresa B, não interferem uma na outra.

Evolução futura

Essa arquitetura também facilita uma futura migração. Se amanhã você substituir o Radmin por uma VPN WireGuard, OpenVPN ou até conexões diretas via internet, basta alterar a forma como a `ConfigEmpresa` é resolvida. O restante da API continua igual, preservando o mesmo contrato com seus clientes.

Considerando toda a arquitetura que você descreveu (Docker na EC2 Linux para a API pública e um gateway Windows para acessar sistemas legados via Radmin), essa

separação entre **API pública** → **Gateway Windows multiempresa** → **Ambiente do cliente** tende a ser uma solução escalável e de baixa manutenção. Ela concentra toda a lógica específica de cada empresa em uma camada de configuração, mantendo o executável único e simplificando tanto o deploy quanto a operação.